# NLP Robot Commander

Commande sûre d’un robot mobile par langage naturel en français

| Document | Rapport technique final |
|---|---|
| Établissement | HESTIM Engineering & Business School |
| Version | Auditée — 22 octobre 2026 |
| Taxonomie | navigate · stop · report |

Résultats locaux : 97,22 % sur validation, 100 % sur le test indépendant et 15/15 scénarios de sûreté conformes.

**La validation Gazebo/Nav2 est explicitement séparée et reste à exécuter sous Ubuntu 22.04.**

# Résumé exécutif

Le projet transforme une phrase française en une décision robotique contrôlée. Le domaine est volontairement fermé à trois intentions : rejoindre un waypoint connu, interrompre une navigation et rapporter la pose actuelle. Classification, extraction, validation, routage et interfaces ROS 2 sont séparés afin que chaque erreur soit localisable.

L'audit a supprimé l'ancienne taxonomie incohérente, reconstruit les jeux de données, durci le chargement du modèle, ajouté des garde-fous pour les négations et commandes composées, et remplacé la fausse confirmation d'arrêt par une machine d'état suivant réellement Nav2.

| Indicateur | Résultat mesuré | Portée |
|---|---|---|
| Validation | 97,22 % accuracy / F1 macro | 36 phrases |
| Test indépendant | 100 % accuracy / F1 macro | 45 phrases |
| Sûreté | 15/15 décisions conformes | cas adversariaux |
| Tests logiciels | 34 réussis | sans ROS 2 réel |

**Aucune conclusion de réussite robotique n'est tirée de ces chiffres. Les waypoints, Gazebo, Nav2, TF et l'immobilisation réelle doivent être vérifiés sur la plateforme cible.**

## Table des matières

1. Périmètre et audit · 2. Architecture · 3. Données · 4. Entraînement · 5. Résultats · 6. Sûreté · 7. ROS 2/Nav2 · 8. Tests · 9. Limites et actions manuelles · 10. Conclusion

# 1. Périmètre et conclusions de l’audit

## Objectifs

- Reconnaître exactement navigate, stop et report.
- Autoriser une navigation uniquement vers un waypoint configuré.
- Donner la priorité à un arrêt explicite non nié.
- Ne jamais inventer une position ou confirmer trop tôt une annulation.

## Constats initiaux

- Les données, résultats et documents utilisaient une taxonomie devenue obsolète.
- Les 14 tests initiaux ne couvraient ni la machine d’état Nav2 ni les refus avancés.
- Le repli lexical silencieux pouvait masquer un checkpoint absent ou corrompu.
- La pose TF était publiée sans contrôle de fraîcheur.
- Le chemin relatif du modèle dépendait du répertoire courant.

## Corrections

La taxonomie a été figée dans le code et le checkpoint, les métriques historiques ont été retirées, le corpus a été reconstruit, le chemin du modèle est explicite et le repli lexical est désactivé par défaut. Les erreurs d’entrée et de modèle sont désormais visibles.

# 2. Architecture logicielle

| Composant | Rôle | Invariant |
|---|---|---|
| CommandProcessor | Validation et politique de sûreté | Aucune action avant acceptation |
| IntentClassifier | Inférence DistilBERT | Trois labels exacts |
| EntityExtractor | Destination et YAML | Une destination unique et finie |
| CommandRouter | Dispatch testable | Une seule opération robot |
| NavigationState | Suivi du but et de l'arrêt | Demande ≠ confirmation |
| NlpCommanderNode | ROS 2, Nav2 et TF | Statuts JSON traçables |

## Flux

Texte → contrôles déterministes → DistilBERT → contrôle confiance/ambiguïté → extraction du waypoint → routeur → ActionClient Nav2 ou lecture TF → statut JSON.

## Interfaces ROS 2

| Nom | Type | Usage |
|---|---|---|
| /nlp_command | std_msgs/String | entrée texte |
| /nlp_commander/status | std_msgs/String JSON | événements |
| /navigate_to_pose | nav2_msgs/NavigateToPose | but et résultat |
| map → base_link | TF2 | pose réelle |
| /cmd_vel | geometry_msgs/Twist | optionnel, désactivé |

# 3. Données et indépendance des jeux

| Ensemble | Taille | Par intention | Usage |
|---|---|---|---|
| Entraînement | 126 | 42 | ajustement des poids |
| Validation | 36 | 12 | sélection du checkpoint |
| Test indépendant | 45 | 15 | mesure finale |
| Sûreté | 15 | mixte | acceptation/refus |

L’audit automatique rapporte 162 phrases de développement, 45 phrases de test, aucun doublon exact et aucun quasi-doublon inter-splits au seuil 0.94.

## Diversité couverte

- Impératifs, questions, politesse, formulations nominales et vocabulaire ROS.
- Alias de cuisine, salon, bureau et chargeur.
- Phrases longues, courtes, techniques et familières.

## Limite

Le corpus reste petit et construit dans un domaine fermé. Le test indépendant est utile, mais son score ne garantit pas la généralisation à toutes les variantes du français.

# 4. Entraînement et chargement du modèle

| Paramètre | Valeur |
|---|---|
| Modèle de base | distilbert-base-multilingual-cased |
| Époques maximales | 12 |
| Batch | 8 |
| Learning rate | 2e-05 |
| Graine | 42 |
| Critère de sélection | F1 macro de validation |
| Longueur maximale | 128 tokens |

Les splits sont figés dans le CSV. Le script configure les graines Python, NumPy et Torch. Le meilleur checkpoint est chargé à la fin de l'entraînement. Seuls les poids sont conservés dans les checkpoints intermédiaires afin d'éviter plusieurs gigaoctets d'états d'optimiseur.

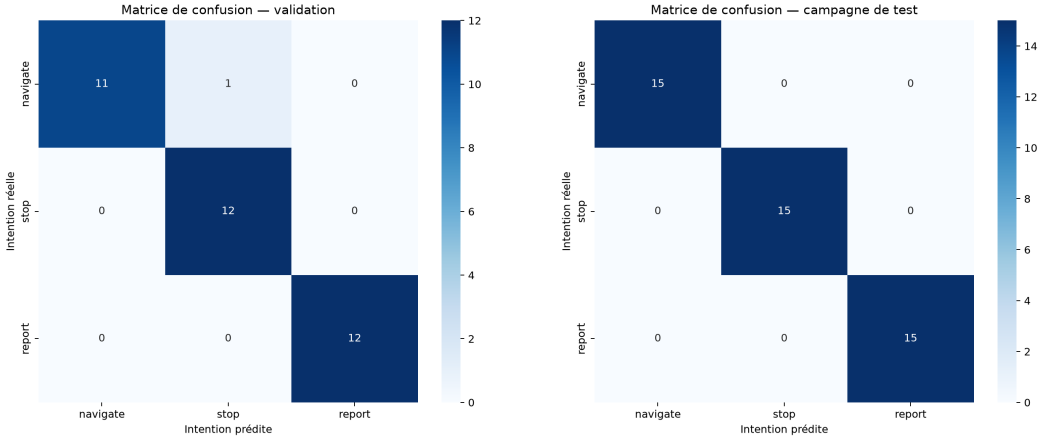
## Chargement défensif

- Répertoire local obligatoire et lisible.
- Tokenizer et poids chargés en mode local pour le runtime robot.
- Ensemble des labels strictement égal aux trois intentions attendues.
- Erreur explicite en cas de dépendance absente ou checkpoint invalide.

**Le checkpoint local dépasse la limite de taille standard de GitHub. Il est reproductible par script mais n'est pas inclus dans le dépôt Git classique.**

# 5. Résultats mesurés

| Jeu | Accuracy | F1 macro | Effectif |
|---|---|---|---|
| Validation | 97.22 % | 97.22 % | 36 |
| Test indépendant | 100.00 % | 100.00 % | 45 |
| Sûreté | 100.00 % | — | 15 |




Matrices de confusion : validation (gauche) et test indépendant (droite).

Le test indépendant produit 45 classifications correctes, avec précision, rappel et F1 égaux à 1,00 pour chaque classe. Les 45 commandes valides sont également acceptées par la politique finale. Ce résultat doit être rapporté avec son effectif et les limites du domaine fermé.

# 6. Sûreté et modes d’échec

| Exemple | Résultat |
|---|---|
| Ne va pas dans la cuisine | NEGATED_COMMAND |
| Va dans la cuisine pas dans le salon | MULTIPLE_DESTINATIONS |
| Va dans la cuisine puis dans le laboratoire | SEQUENTIAL_COMMAND |
| Arrête-toi immédiatement | arrêt prioritaire accepté |
| Va dans la chambre | UNKNOWN_DESTINATION |
| Bonjour robot | UNKNOWN_INTENT |
| Commande avec contrôle Unicode | CONTROL_CHARACTER |

Les 15 scénarios adversariaux produisent la décision attendue avec le checkpoint. Un score de réseau élevé n’est toutefois jamais utilisé seul : les preuves lexicales et les entités doivent rester compatibles avec l’action.

## Catalogue opérationnel

- LOW_CONFIDENCE et AMBIGUOUS_INTENT : refus sans routage.
- navigation_rejected : serveur absent, but refusé ou erreur d’envoi.
- stop_unconfirmed : annulation refusée, expirée ou résultat incompatible.
- position_unavailable / position_stale : aucune pose présentée comme actuelle.

# 7. Intégration ROS 2, Nav2 et TF

## Navigation

Le nœud attend `/navigate_to_pose` avec timeout, construit une pose dans `map`, envoie le but asynchrone puis publie séparément demande, acceptation et résultat. Un second but est refusé tant que la machine d'état n'est pas revenue à idle.

## Arrêt

| Étape | Événement | Signification |
|---|---|---|
| 1 | stop_received | demande reçue |
| 2 | cancellation_requested | requête envoyée à Nav2 |
| 3 | cancellation_accepted | Nav2 traite la requête |
| 4a | stop_confirmed | résultat canceled ou aucun but actif |
| 4b | stop_unconfirmed | refus, erreur, timeout ou résultat incompatible |

L'envoi optionnel d'une vitesse nulle est désactivé par défaut et ne vaut jamais confirmation. Il doit être coordonné avec le multiplexeur de vitesse du robot.

## Rapport

La transformation `map → base_link` est lue avec timeout. Son timestamp est comparé à `max_tf_age_sec`. Une transformation nulle, future ou trop ancienne est refusée.

# 8. Tests exécutés et non exécutés

## Exécutés

- 34 tests unitaires réussis.
- Compilation Python de scripts, sources et tests réussie.
- Audit des datasets : ok, sans fuite détectée au seuil configuré.
- Entraînement DistilBERT et sélection du meilleur checkpoint.
- Évaluation indépendante 45/45 et sûreté 15/15.
- Validation structurelle de la présentation et rendu page par page du PDF.

## Non exécutés

- Construction `colcon` sur Ubuntu 22.04.
- Gazebo, RViz2 et lifecycle Nav2.
- Navigation réelle vers les quatre waypoints.
- Annulation d’un but actif et immobilisation observée.
- TF absente, trop ancienne et comparaison de pose avec RViz2.
- Entrée microphone dans un environnement bruité.

**Ces éléments figurent dans ACTIONS_MANUELLES.md et checklist_demo.md. Ils ne doivent pas être présentés comme validés avant collecte des preuves.**

# 9. Limites, recommandations et remise

## Limites

- Petit corpus et domaine fermé malgré l’indépendance du test.
- Waypoints d’exemple non liés à une carte validée.
- Absence de mesure de latence et de calibration probabiliste.
- Poids DistilBERT non stockés dans Git.
- Audio non évalué dans la campagne actuelle.

## Recommandations

- Exécuter plusieurs graines et publier moyenne et dispersion.
- Ajouter un jeu de test collecté auprès d’utilisateurs externes.
- Mesurer la latence CPU du robot et la fraîcheur TF en situation réelle.
- Archiver les journaux d’action Nav2 avec chaque vidéo de démonstration.
- Utiliser Git LFS ou un registre de modèles si les poids doivent être distribués.

## Commandes principales

`python -m pytest` · `python scripts/audit_dataset.py` · `python scripts/train_intent_classifier.py ...` · `ros2 launch nlp_robot_commander simulation.launch.py model_path:=... map:=...`

# 10. Conclusion

La version auditée atteint l’objectif NLP sur les jeux fournis et renforce les interfaces robotique critiques. La taxonomie est cohérente, les données sont auditées, le checkpoint est reproductible, les refus sont explicites et la sémantique de l’arrêt ne confond plus demande et confirmation.

Le projet est prêt pour sa dernière phase : recalibrer les waypoints, construire sous ROS 2 Humble, exécuter la campagne Gazebo/Nav2 et intégrer uniquement les preuves observées.

## Livrables de référence

- README.md — reproduction et lancement.
- docs/rapport_final.md — source du présent rapport.
- docs/guide_utilisateur.md — prise en main.
- docs/checklist_demo.md — déroulé de soutenance.
- docs/ACTIONS_MANUELLES.md — validations restantes.
- artifacts/ — métriques, prédictions et matrices.